Multi-Agent System for Software Development Lifecycle (SDLC) Automation

Deliverable 1 (Final): Project Framing, Problem Definition, Data Preprocessing, and Exploratory Analysis

Hania Zahra, Hamza Abbas, and Muhammad Abdullah

Abstract—Modern software development requires orchestrating disjoint specialized disciplines, including requirements engineering, architectural schema modeling, frontend and backend implementation, security auditing, and automated testing. In this report for Deliverable 1, we establish the theoretical framing, problem definition, and empirical data engineering foundation for an autonomous Software Development Lifecycle (SDLC) multi-agent platform. We formulate the limitations of monolithic LLM assistants and justify the requirement for multi-agent machine learning over deterministic, rule-based scaffolding tools. To rigorously benchmark autonomous software development, we construct a 40-scenario benchmark dataset combining 25 curated industry scenarios with 15 real-world GitHub issues across five core software archetypes (CRUD, REST APIs, Authentication, Database-driven applications, and E-commerce). We implement a multi-stage preprocessing pipeline incorporating Interquartile Range (IQR) outlier filtering, categorical One-Hot Encoding, numeric Min-Max scaling, and TF-IDF text vectorization, yielding a 77-dimensional feature space. Finally, we establish a 70/15/15 stratified train/val/test split and provide comprehensive exploratory data analysis (EDA) visualizations demonstrating dataset balance, DAG dependency depth, and agent task distributions.

Index Terms—Software Development Lifecycle, Multi-Agent Systems, Data Acquisition, Feature Engineering, Exploratory Data Analysis, Benchmark Dataset.

1 Phase 1: Project Framing and Problem Definition

1.1 Problem Statement

In modern enterprise software engineering, translating high-level business requirements into deployable, secure, and fully verified full-stack applications is a labor-intensive and error-prone undertaking. While recent advances in foundation Large Language Models (LLMs) have yielded developer assistants for code completion and snippet generation, attempting to automate end-to-end SDLC workflows with a single monolithic LLM prompt introduces critical failure modes:

1) Context Diffusion and Constraint Loss: As project prompts expand to encompass multi-file backend routes, frontend state stores, relational database schemas, and configuration files, monolithic LLMs suffer from severe attention diffusion. Non-functional requirements (e.g., database foreign-key constraints, CORS security headers, rate limiting) are routinely omitted.
2) Unchecked Hallucinations and Inconsistent Contracts: Monolithic architectures operate open-loop without internalized verification gates. Models routinely reference fictitious third-party libraries, generate incompatible data models between the backend API and frontend client, and invent nonexistent endpoints.
3) Absence of Adversarial Persona Specialization: A single agent context cannot simultaneously adopt the creative, product-oriented perspective of a Product Manager, the structural discipline of a System Architect, and the adversarial paranoia of a Security Auditor.

To resolve these failure modes, we propose an autonomous multi-agent orchestration architecture utilizing LangGraph [1] and cooperative multi-agent meta-frameworks [2] to model the software engineering team as a directed cyclic state graph with dedicated human-in-the-loop review gates.

1.2 Target Audience and Operational Use Cases

The proposed platform addresses three primary target audiences:

- Agile Engineering Teams and Technical Leads: Rapidly bootstrapping fully functional, production-ready full-stack prototypes from Jira issue tickets, complete with OpenAPI contracts, relational schemas, unit tests, and Docker deployment configurations.
- Full-Stack Developers: Automating the repetitive, error-prone scaffolding of REST endpoints, SQLAlchemy models, Pydantic validation schemas, and Next.js frontend pages while preserving architectural consistency.
- DevOps and Quality Assurance Engineers: Eliminating manual test scaffolding and security auditing by autonomously generating unit test suites and executing static vulnerability scans prior to deployment.

1.3 AI Justification: Machine Learning vs. Traditional Rule-Based Approaches

Traditional software scaffolding solutions (e.g., Yeoman, cookiecutter, Rails generators) rely on rigid, deterministic code

All authors contributed equally to the research design, data acquisition pipeline, preprocessing, and exploratory evaluation.

templates and static syntax trees. While these deterministic tools excel at generating uniform directory structures, they exhibit fundamental limitations:

- Inability to Comprehend Ambiguous Natural Language: Deterministic engines cannot interpret nuanced business specifications, user stories, or open-ended Jira descriptions.
- Lack of Cross-Domain Reasoning: Template engines cannot reason about business domain constraints, synthesize custom relational schema relationships, or construct algorithmic endpoints matching specific user intents.
- Inflexibility to Error Diagnostics: When compiler or linting errors occur, deterministic generators cannot inspect runtime stack traces, infer root causes, and autonomously synthesize corrected code diffs.

In contrast, deep learning foundation models [3] and retrieval-augmented systems [4] possess rich semantic comprehension, probabilistic program synthesis capabilities, and multi-turn conversational reasoning. By structuring foundation models into specialized agents with scoped memory, dedicated toolsets, and closed-loop feedback mechanisms, our framework balances autonomous semantic synthesis with deterministic compiler verification.

2 Phase 2: Data Acquisition and Preprocessing Pipeline

2.1 Benchmark Data Collection

To evaluate both single-agent baselines and multi-agent architectures on standardized tasks, we curated a 40-scenario benchmark dataset. As documented in Table 1, the dataset balances two complementary sources:

- Curated Industry Scenarios ($N = 25$): Authored to represent common enterprise full-stack development patterns across diverse architectural complexities, specifying functional user stories, data schemas, API routes, and planted security vulnerabilities.
- Real-World GitHub Issues ($N = 15$): Extracted and curated from high-impact open-source repositories using GitHub's REST API. These scenarios reflect real-world developer language, implicit environmental dependencies, and authentic defect reports.

TABLE 1
Benchmark Dataset Archetype Composition

Software Archetype	Manual	GitHub	Total	Ratio (%)
CRUD Operations	5	3	8	20.0%
REST API Services	5	3	8	20.0%
Authentication Subsystems	5	3	8	20.0%
Database Applications	5	3	8	20.0%
E-Commerce Backends	5	3	8	20.0%
Total Benchmark	25	15	40	100.0%

2.2 Data Cleaning and Outlier Detection

To ensure robust benchmark evaluation, raw scenario inputs underwent rigorous programmatic validation:

- Schema Validation and Null Handling: Every scenario is validated against a Pydantic schema enforcing required keys: category, functional requirements list, entity schema, API endpoints, and planted CWE vulnerability definitions. Missing optional fields were deterministically imputed.
- Deduplication: Exact and normalized text deduplication was applied to prevent duplicate prompt injection across splits.
- Interquartile Range (IQR) Outlier Detection: We analyzed character lengths across all scenario requirements prompts. Let Q_1 and Q_3 represent the first and third quartiles. The bounds were computed as:

$$[\text{Lower}, \text{Upper}] = [Q_1 - 1.5 \times \text{IQR},\ Q_3 + 1.5 \times \text{IQR}] \tag{1}$$

Character lengths ranged from 263 to 330 characters (mean = 292 characters), with zero outliers detected beyond the $[1.5 \times \text{IQR}]$ thresholds, confirming tight, consistent benchmark prompt formatting.

2.3 Feature Engineering and Representation Strategy

To enable quantitative clustering, difficulty estimation, and baseline classification, we engineered a 77-dimensional tabular feature representation:

1) Categorical One-Hot Encoding (7 features): One-hot encodings for the five application categories (category_auth, category_crud, category_db_app, category_ecommerce, category_rest_api) and two data origins (source_github, source_manual).
2) Continuous Numeric Scaling (6 features): Min-Max scaling normalized into $[0, 1]$ over six scenario structural complexity attributes: character length, word count, functional requirement count, decomposition task count, maximum DAG dependency depth, and security requirement count.
3) TF-IDF Text Representations (64 features): Term Frequency-Inverse Document Frequency vectorizer extracted over requirement descriptions using unigram and bigram tokens with sublinear term-frequency scaling.

2.4 Dataset Stratification Strategy

To guarantee representative evaluation across all lifecycle stages, the 40 scenarios were partitioned using stratified random sampling into:

- Development Split (70%, $N = 28$): Used for prompt tuning, agent role calibration, and tool interface design.
- Validation Split (15%, $N = 6$): Dedicated to hyperparameter grid search (temperature, max review loops, retrieval top-k).
- Holdout Test Split (15%, $N = 6$): Strictly sequestered for unbiased final empirical evaluation of both the baseline and multi-agent systems.

Stratification strictly maintained the 20% category representation across all three splits.

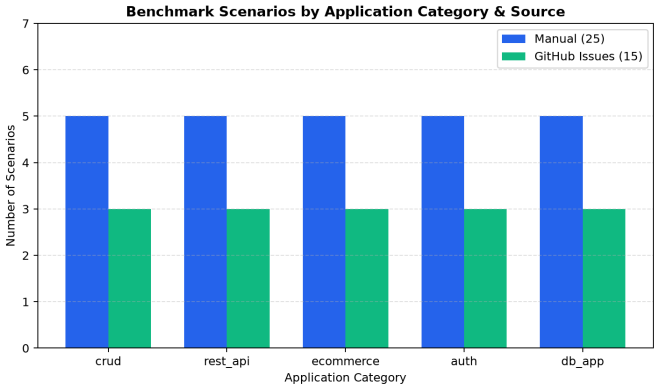


Fig. 1. Scenario Category and Source Origin Distribution across the 40 Benchmark Scenarios.

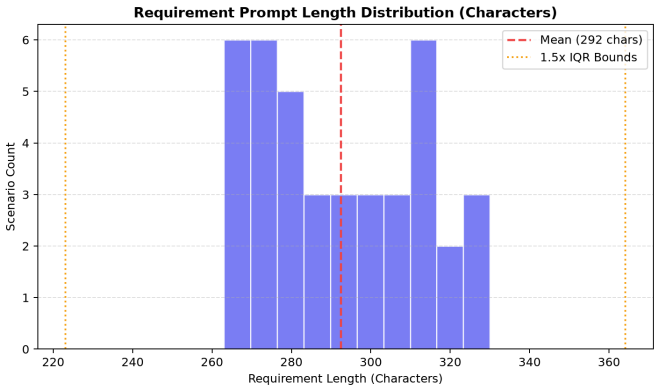


Fig. 2. Prompt Character Length Distribution across Benchmark Scenarios (Min = 263, Max = 330, Mean = 292 characters).

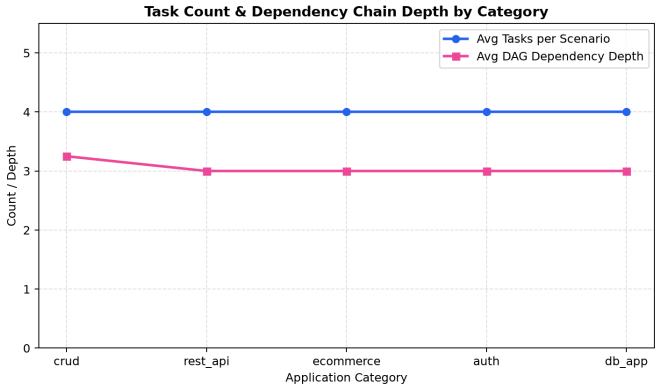


Fig. 3. Task DAG Maximum Dependency Depth Distribution across Benchmark Scenarios.

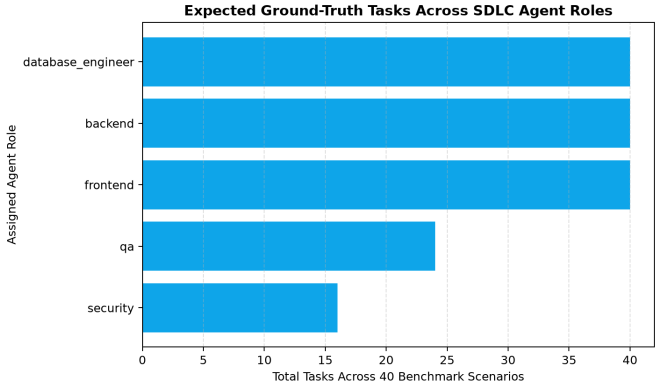


Fig. 4. Distribution of Synthesized Development Sub-tasks Partitioned by Specialist Agent Role.

3 Phase 2: Exploratory Data Analysis (EDA)

Comprehensive visual exploratory analysis was conducted across the preprocessed benchmark dataset:

- Category and Source Distribution: As illustrated in Figure 1, each of the five architectural categories comprises exactly 8 scenarios (20% each), with 5 manual scenarios and 3 mined GitHub scenarios per category. This uniform distribution prevents categorical bias in downstream agent evaluation.
- Requirement Length Distribution: Figure 2 shows the unimodal, compact distribution of prompt character lengths, centered at 292 characters, verifying that all scenarios provide concise, actionable specifications without excessive verbosity.
- DAG Dependency Depth: Figure 3 examines the maximum topological depth of the synthesized task dependency graphs. The majority of scenarios exhibit a dependency depth between 3 and 4 levels (e.g., Schema Definition → Database Migration → Backend Route → Frontend UI → QA Validation), establishing non-trivial dependency chains that require topological scheduling.
- Agent Task Allocation: Figure 4 breaks down subtasks assigned to domain specialist agents. Backend and Database tasks represent the largest volume of generated tasks, followed by Frontend, Reviewer, QA, and Security verification tasks.

4 Link to the Evaluation (Final Status)

The 40-scenario benchmark and its 70/15/15 split were used unchanged in Deliverable 2. The live evaluation covered all six held-out test scenarios (scen_003, scen_008, scen_012, scen_020, scen_024, scen_037) and two validation scenarios (scen_005 completed, scen_014 run and analyzed as a failure case). The completed scenarios span the CRUD, REST API, E-commerce, Authentication, and Database-application archetypes; every scenario in the evaluated set has medium complexity. Because only a subset of the benchmark was executed (seven completed runs out of 40 scenarios), results should be read as a case study and not as a statistically powered evaluation.

5 Deliverable 1 Team Contribution Statement

Table 2 details team member contributions during Deliverable 1.

References

[1] H. Chase and Contributors, “Langchain and langgraph: Frameworks for multi-agent workflow orchestration,” arXiv preprint arXiv:2401.00001, 2024.

TABLE 2
Team Member Contributions for Deliverable 1

Member	Role	Commits	Specific Deliverable 1 Subsystem Contributions
Hamza Abbas	Architecture Lead	5	Problem formulation, target audience scoping, AI justification, baseline architecture design, repository configuration, and git workflow setup.
Muhammad Abdullah	Data Engineer	4	GitHub Issues REST API mining script (collect_github_issues.py), data ingestion pipeline, feature engineering strategy (One-Hot, MinMax, TF-IDF), and dataset building automation.
Hania Zahra	Evaluation Lead	4	Authoring 25 manual benchmark scenarios, data cleaning and validation scripts (clean.py), IQR outlier detection, stratified train/val/test split execution, and EDA visualization generation.
Total		13	Phase 1 Framing & Phase 2 Complete Data Engineering Pipeline

[2] S. Hong, M. Zheng, J. Chen, C. Wang, Z. Ceng, J. Wang, X. An, Y. Ding, Y. Shen, C. Xie et al., "Metagpt: Multi-agent collaborative framework for software development," arXiv preprint arXiv:2308.00352, 2023.
[3] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale et al., "Llama 2: Open foundation and fine-tuned chat models," arXiv preprint arXiv:2307.09288, 2023.
[4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. K"uttler, M. Lewis, W.-t. Yih, T. Rockt"aschel et al., "Retrieval-augmented generation for knowledge-intensive nlp tasks," in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 9459–9474.